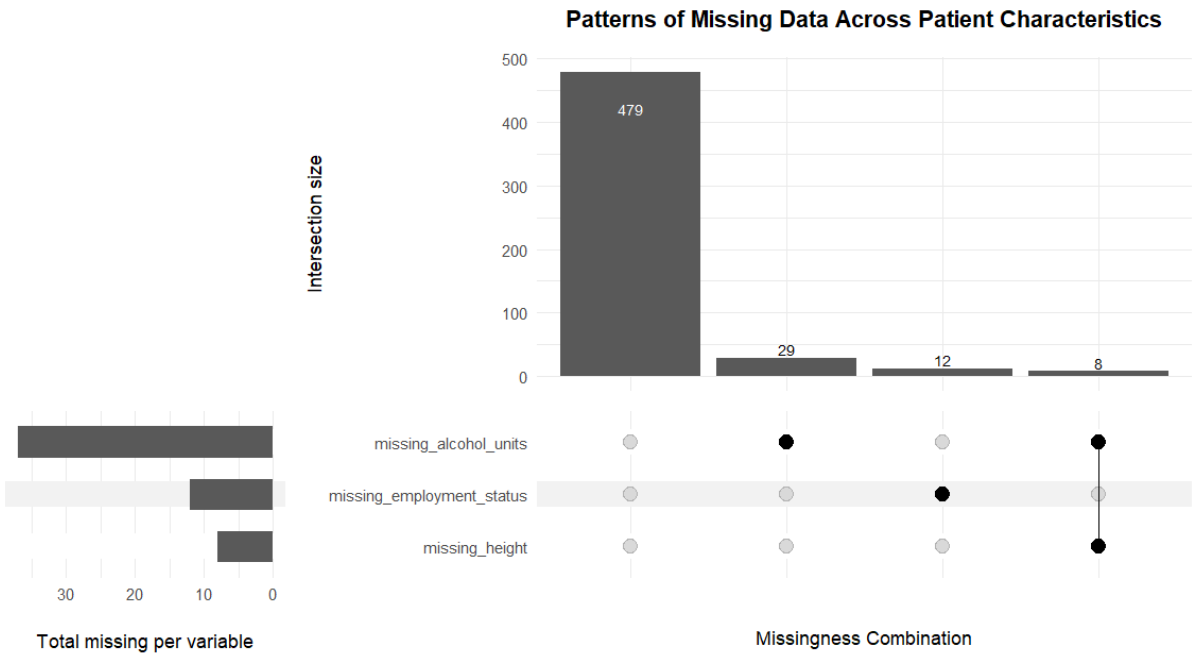
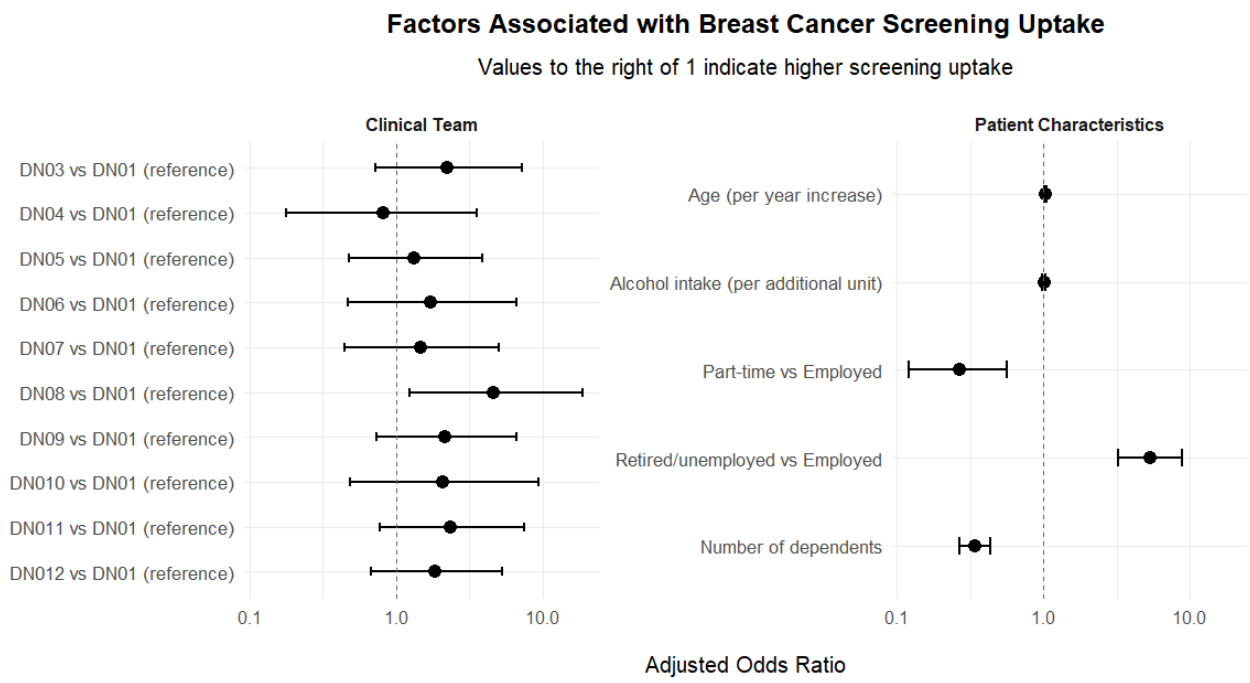
Appendix 1 – Supplementary Figures



Supplementary Figure 1. Distribution of missing data across patient characteristics (N = 528). Bars represent the number of patients with each pattern of missing variables. Missingness was predominantly limited to single characteristics, with very few patients missing multiple variables, suggesting isolated recording omissions rather than systematic gaps in data collection.



Supplementary Figure 2. Adjusted odds ratios for factors associated with breast cancer screening uptake. Forest plot showing adjusted odds ratios (ORs) and 95% confidence intervals from the primary multivariable logistic regression model. The model included age, alcohol intake, employment status, number of dependents, and clinical team. Clinical teams are shown relative to DN01 (reference). Odds ratios greater than 1 indicate higher likelihood of screening attendance, and values less than 1 indicate lower likelihood. The vertical dashed line represents no association (OR = 1).

Appendix 2 – Analytical Methods and Code

0.1 Overview

This document contains the reproducible R code used for data discovery, cleaning, validation, exploratory analysis, and statistical modelling of breast cancer screening uptake across 12 clinical team datasets. Scripts are presented in the order executed.

0.2 Computational Environment

The following R packages were used in the analysis (versions shown below):

```
##    tidyverse       naniar ComplexUpset   effectsize        binom        broom
##      "2.0.0"      "1.1.0"      "1.3.3"      "1.0.1"    "1.1.1.1"     "1.0.12"
##       scales      janitor        skimr          car  sessioninfo
##      "1.4.0"      "2.2.1"      "2.2.2"      "3.1.5"      "1.2.3"
```

All analyses were conducted in R (version 4.5.2, 2025-10-31 ucrt) using RStudio (version 2026.01.0+392) on a Windows 10 x64 (build 19045) operating system.

0.3 Project Directory Structure

```
##
## Costello_Medical_Screening_Analysis/
##    raw_data/
##       DN01.csv
##       DN02.csv
##       ...
##    output/
##       breast_screening_cleaned.csv
##       Missing_values_per_variable.png
##       number_of_dependents_by_clinical_team.png
##       ...
##    scripts/
##       01_data_discovery.R
##       02_data_cleaning.R
##       03_validation.R
##       04_exploratory_data_analysis.R
##       05_statistical_modelling.R
##       Costello Medical Screening Analysis in R.Rmd
##    report/
##       Report.docx
##       Costello-Medical-Screening-Analysis-in-R.pdf
##    Costello_Medical_Screening_Analysis.Rproj
```

0.4 Data Discovery (01_data_discovery.R)

```r
# ============================================================
# Script: 01_data_discovery.R
# Purpose: Initial exploration of raw screening datasets
#          prior to cleaning and harmonisation.
# Author: Nikolaos Samperis
# Date: 20/02/2026
# ============================================================

# ------------------------------------------------------------
# 1. Load required libraries
# ------------------------------------------------------------
# tidyverse: data manipulation and iteration
# stringr:   numeric-aware sorting of file names

library(tidyverse)
library(stringr)

# ------------------------------------------------------------
# 2. Identify and order raw CSV files
# ------------------------------------------------------------
# Assumes all raw datasets are stored in a folder named
# "raw_data". Files are sorted numerically to preserve the
# intended dataset order (e.g., DN01, DN02, ...).

files <- list.files("raw_data",
                    pattern = "\\.csv$",
                    full.names = TRUE) |>
  str_sort(numeric = TRUE)

# ------------------------------------------------------------
# 3. Import all datasets into a named list
# ------------------------------------------------------------
# Each dataset is read as a tibble.
# Names are assigned using the file names for traceability.

datasets <- set_names(files, basename(files)) |>
  map(~ read_csv(.x, show_col_types = FALSE))

# ------------------------------------------------------------
# 4. Structural inspection of each dataset
# ------------------------------------------------------------
# Displays variable names, types, and preview of values.
# Used to detect inconsistencies in structure across files.

walk2(datasets, names(datasets), ~ {
```

```r
  cat("\n\n========================\n")
  cat("Dataset:", .y, "\n")
  cat("========================\n")
  glimpse(.x)
})

# ----------------------------------------------------------
# 5. Check variable names across datasets
# ----------------------------------------------------------
# Used to identify naming inconsistencies prior to cleaning.

map(datasets, names)

# ----------------------------------------------------------
# 6. Check variable data types
# ----------------------------------------------------------
# Identifies potential inconsistencies (e.g., numeric vs character)
# across datasets that may require harmonisation.

map_dfr(datasets,
        ~ summarise_all(.x, class),
        .id = "dataset")

# ----------------------------------------------------------
# 7. Dataset-level dimensions and missingness
# ----------------------------------------------------------
# For each dataset:
#   - number of rows
#   - number of columns
#   - total number of missing values
#   - percentage of missing values
# This provides an overview of completeness by file.

map_dfr(datasets, ~ tibble(
  rows = nrow(.x),
  cols = ncol(.x),
  total_missing = sum(is.na(.x)),
  missing_pct = mean(is.na(as.matrix(.x))) * 100
), .id = "dataset")

# ----------------------------------------------------------
# 8. Missing values per variable within each dataset
# ----------------------------------------------------------
# Summarises number of missing values per column for each dataset.
# Used to detect systematic missingness patterns.

map_dfr(datasets, ~
```

```r
          summarise(.x,
                    across(everything(),
                           ~ sum(is.na(.)))),
        .id = "dataset")

# ----------------------------------------------------------
# 9. Identify duplicated rows within each dataset
# ----------------------------------------------------------
# Flags fully duplicated records (forward or backward).
# No removal is performed at this stage - this is exploratory.

walk2(datasets, names(datasets), ~ {

  dup_rows <- .x[duplicated(.x) | duplicated(.x, fromLast = TRUE), ]

  if(nrow(dup_rows) > 0){
    cat("\nDuplicated rows in:", .y, "\n")
    print(dup_rows)
  }
})

# ----------------------------------------------------------
# 10. Summary statistics for numeric variables
# ----------------------------------------------------------
# Calculates min, median, mean, and max for numeric variables
# in each dataset.
# Used to detect:
#   - Implausible values
#   - Data entry errors
#   - Outliers
#   - Distribution differences between datasets

map_dfr(datasets, ~
          summarise(.x,
                    across(where(is.numeric),
                           list(
                             min = ~ min(.x, na.rm = TRUE),
                             median = ~ median(.x, na.rm = TRUE),
                             mean = ~ mean(.x, na.rm = TRUE),
                             max = ~ max(.x, na.rm = TRUE)
                           ))),
        .id = "dataset")
```

0.5 Data Cleaning and Harmonisation (02_data_cleaning.R)

```r
# ==========================================================
# Script: 02_data_cleaning.R
# Purpose: Clean, harmonise, and combine raw screening datasets
#          into a single analysis-ready dataset.
# Author: Nikolaos Samperis
# Date: 20/02/2026
# ==========================================================


# ----------------------------------------------------------
# 1. Load required libraries
# ----------------------------------------------------------
# tidyverse: data manipulation
# stringr:   file name processing

library(tidyverse)
library(stringr)

# ----------------------------------------------------------
# 2. Load and order raw datasets
# ----------------------------------------------------------
# Ensures consistent dataset ordering (DN01-DN012).

files <- list.files("raw_data",
                    pattern = "\\.csv$",
                    full.names = TRUE) |>
  str_sort(numeric = TRUE)

datasets <- set_names(files, basename(files)) |>
  map(~ read_csv(.x, show_col_types = FALSE))

# ----------------------------------------------------------
# 3. Remove auto-generated index column
# ----------------------------------------------------------
# Some CSV files contain an export index column ("...1").
# This is not analytically meaningful and is removed.

datasets <- map(datasets, ~ {
  if(names(.x)[1] == "...1") {
    select(.x, -1)
  } else {
    .x
  }
})
```

```r
# ----------------------------------------------------------
# 4. Standardise variable names
# ----------------------------------------------------------
# Variable names are harmonised across datasets to ensure
# consistent naming prior to merging.

datasets <- map(datasets, ~ {

  df <- .x
  names_lower <- tolower(names(df))

  # Rename based on lowercase comparison
  if("screening" %in% names_lower)
    df <- rename(df, screening_decision = !!names(df)[which(names_lower == "screening")])

  if("age" %in% names_lower)
    df <- rename(df, age = !!names(df)[which(names_lower == "age")])

  if("weight" %in% names_lower)
    df <- rename(df, weight = !!names(df)[which(names_lower == "weight")])

  if("height" %in% names_lower)
    df <- rename(df, height = !!names(df)[which(names_lower == "height")])

  if("alcohol" %in% names_lower)
    df <- rename(df, alcohol_units = !!names(df)[which(names_lower == "alcohol")])

  if("es" %in% names_lower)
    df <- rename(df, employment_status = !!names(df)[which(names_lower == "es")])

  if("deps" %in% names_lower)
    df <- rename(df, n_dependents = !!names(df)[which(names_lower == "deps")])

  df
})

# ----------------------------------------------------------
# 5. Ensure structural consistency (DN01 issue)
# ----------------------------------------------------------
# DN01 does not contain employment_status.
# Add missing column to preserve consistent structure.

datasets <- map(datasets, ~ {
  if(!"employment_status" %in% names(.x)) {
    .x$employment_status <- NA
  }
  .x
```

```r
})

# ----------------------------------------------------------
# 6. Standardise column order
# ----------------------------------------------------------
# Enforces identical column ordering across datasets.

standard_cols <- c(
  "age",
  "weight",
  "height",
  "alcohol_units",
  "employment_status",
  "n_dependents",
  "screening_decision"
)

datasets <- map(datasets, ~ .x[, standard_cols])

# ----------------------------------------------------------
# 7. Type conversion and unit harmonisation
# ----------------------------------------------------------
# Convert variables to appropriate types.
# Height converted from centimetres to metres for analysis.

datasets <- map(datasets, ~ {

  .x |>
    mutate(
      age = as.numeric(age),
      weight = as.numeric(weight),
      height = as.numeric(height) / 100,   # cm → meters (no rounding)
      alcohol_units = as.numeric(alcohol_units),
      n_dependents = as.integer(n_dependents),
      employment_status = as.factor(employment_status),
      screening_decision = as.factor(screening_decision)
    )
})

# ----------------------------------------------------------
# 8. Add clinical team identifier and merge
# ----------------------------------------------------------
# Adds dataset source identifier before row-binding.


combined_data <- imap_dfr(
  datasets,
```

```r
  ~ mutate(.x,
           clinical_team = str_remove(.y, "\\.csv$"))
)


combined_data$clinical_team <- factor(combined_data$clinical_team)

# ----------------------------------------------------------
# 9. Final validation checks
# ----------------------------------------------------------
# Verify structure and basic summaries of merged dataset.

glimpse(combined_data)
summary(combined_data)

# Confirm height now expressed in metres
summary(combined_data$height)

# ----------------------------------------------------------
# 10. Export cleaned dataset
# ----------------------------------------------------------
# Round numeric display precision at export stage only
# (avoids floating-point artefacts while preserving accuracy).

combined_data <- combined_data |>
  mutate(
    height = round(height, 3),
    weight = round(weight, 3)
  )

if(!dir.exists("output")) {
  dir.create("output")
}

write_csv(combined_data,
          file = "output/breast_screening_cleaned.csv")
```

0.6 Data Validation (03_validation.R)

```r
# ==========================================================
# Script: 03_validation.R
# Purpose: Perform structural and logical validation checks
#          on the cleaned breast screening dataset.
# Author: Nikolaos Samperis
# Date: 20/02/2026
```

```r
# ============================================================

# ------------------------------------------------------------
# 1. Load required libraries and dataset
# ------------------------------------------------------------
# tidyverse: data manipulation
# janitor:   column name validation
# skimr:     structured dataset summary

library(tidyverse)
library(janitor)
library(skimr)

# Import cleaned dataset with explicit column types
# (ensures structural consistency at validation stage)

data <- read_csv(
  "output/breast_screening_cleaned.csv",
  col_types = cols(
    age = col_double(),
    weight = col_double(),
    height = col_double(),
    alcohol_units = col_double(),
    n_dependents = col_integer(),
    employment_status = col_factor(),
    screening_decision = col_factor(),
    clinical_team = col_factor()
  )
)

cat("Validation started...\n\n")

# ------------------------------------------------------------
# 2. Structural validation
# ------------------------------------------------------------
# Confirm dataset dimensions and column integrity.

cat("===== STRUCTURE =====\n")
cat("Rows:", nrow(data), "\n")
cat("Columns:", ncol(data), "\n\n")

# Ensure column names follow clean naming conventions
if(any(names(data) != make_clean_names(names(data)))) {
  warning("Column names are not fully standardised.")
}

# ------------------------------------------------------------
```

```r
# 3. Duplicate record assessment
# ---------------------------------------------------------
# Identify fully duplicated rows (including first occurrence).

cat("===== DUPLICATE CHECK =====\n")

duplicate_rows <- data[
  duplicated(data) | duplicated(data, fromLast = TRUE),
]

duplicate_count <- nrow(duplicate_rows)

cat("Number of duplicated rows:", duplicate_count, "\n")

if(duplicate_count == 0){
  cat("No duplicated rows found.\n\n")
} else {
  cat("Duplicated rows detected. Displaying all duplicates:\n")
  print(duplicate_rows)
  cat("\n")
}

# ---------------------------------------------------------
# 4. Missing data validation
# ---------------------------------------------------------
# Assess missingness overall and by clinical team
# to detect potential systematic patterns.

if("clinical_team" %in% names(data)){

  cat("===== MISSINGNESS BY CLINICAL TEAM =====\n")

  vars_to_check <- setdiff(names(data), "clinical_team")

  missing_by_team <- data %>%
    group_by(clinical_team) %>%
    summarise(
      across(
        all_of(vars_to_check),
        list(
          missing_count  = ~ sum(is.na(.)),
          missing_percent = ~ round(mean(is.na(.)) * 100, 2)
        )
      ),
      .groups = "drop"
    ) %>%
    pivot_longer(
```

```r
      cols = -clinical_team,
      names_to = c("variable", ".value"),
      names_pattern = "(.+)_(missing_count|missing_percent)"
    ) %>%
    filter(missing_count > 0) %>%
    arrange(desc(missing_percent))

  if(nrow(missing_by_team) == 0){
    cat("No missing values detected across clinical teams.\n\n")
  } else {
    print(missing_by_team, n = Inf)
    cat("\n")
  }
}

# ----------------------------------------------------------
# 5. Logical plausibility checks
# ----------------------------------------------------------
# Evaluate whether key numeric variables fall within
# clinically plausible ranges.

cat("===== LOGICAL CHECKS =====\n")

# Age plausibility (adult screening population)
if("age" %in% names(data)){
  implausible_age <- sum(data$age < 18 | data$age > 100, na.rm = TRUE)
  cat("Implausible age values (<18 or >100):", implausible_age, "\n")
}

# Height plausibility (in metres)
if("height" %in% names(data)){
  implausible_height <- sum(data$height < 1.3 | data$height > 2.2, na.rm = TRUE)
  cat("Implausible height values (<1.3m or >2.2m):", implausible_height, "\n")
}

# Weight plausibility (in kg)
if("weight" %in% names(data)){
  implausible_weight <- sum(data$weight < 30 | data$weight > 250, na.rm = TRUE)
  cat("Implausible weight values (<30kg or >250kg):", implausible_weight, "\n")
}

cat("\n")

# ----------------------------------------------------------
# 6. Overall dataset profile
# ----------------------------------------------------------
# Generate structured summary of variable distributions
```

```r
# to support exploratory interpretation.

cat("===== SKIM SUMMARY =====\n")
skim(data)

cat("\nValidation completed successfully.\n")
```

0.7 Exploratory Data Analysis (04_exploratory_data_analysis.R)

```r
# ==========================================================
# Script: 04_exploratory_analysis.R
# Purpose: Perform exploratory and descriptive analysis of
#          the cleaned breast screening dataset.
# Author: Nikolaos Samperis
# Date: 20/02/2026
# ==========================================================


# ----------------------------------------------------------
# 1. Load required libraries
# ----------------------------------------------------------
# tidyverse: data manipulation & visualisation
# naniar: missing data visualisation
# ComplexUpset: missingness pattern plots
# effectsize: effect size calculations
# binom: binomial confidence intervals
# broom: model tidying

library(tidyverse)
library(naniar)
library(ComplexUpset)
library(effectsize)
library(binom)
library(broom)

# ----------------------------------------------------------
# 2. Load cleaned dataset
# ----------------------------------------------------------

# Ensure categorical variables are correctly specified
data <- data |>
  mutate(
    clinical_team = factor(clinical_team),
    screening_decision = factor(screening_decision),
    employment_status = factor(employment_status)
```

```r
  )

# ----------------------------------------------------------
# 3. Overall missingness assessment
# ----------------------------------------------------------
# Summarise total missing values per variable.

missing_summary <- data.frame(
  variable = names(data),
  missing_count = colSums(is.na(data)),
  missing_percent = round(colMeans(is.na(data)) * 100, 2)
) |>
  arrange(desc(missing_percent))

print("===== OVERALL MISSINGNESS =====")
print(missing_summary)

# Visualisation of missingness per variable
p <- gg_miss_var(data)

for (i in seq_along(p$layers)) {
  g <- class(p$layers[[i]]$geom)[1]

  if (g == "GeomPoint") {
    p$layers[[i]]$aes_params$size <- 3
  }

  if (g == "GeomBar") {
    p$layers[[i]]$aes_params$linewidth <- 1.5
    p$layers[[i]]$aes_params$size <- 1.5
  }
}

print(
  p +
    labs(
      title = "Missing Values per Recorded Patient Characteristic",
      y = "Number of Missing Observations",
      x = NULL
    ) +
    theme(
      plot.title = element_text(face = "bold", margin = margin(b = 20), hjust = 0.5),
      axis.title.x = element_text(margin = margin(t = 20)),
      legend.position = "none"
    )
)
```

```r
# ----------------------------------------------------------
# 4. Missingness pattern (UpSet)
# ----------------------------------------------------------
# Examine joint missingness patterns across key variables.

data_upset <- data |>
  mutate(
    missing_alcohol_units     = is.na(alcohol_units),
    missing_employment_status = is.na(employment_status),
    missing_height            = is.na(height)
  )

upset(
  data_upset,
  intersect = c(
    "missing_alcohol_units",
    "missing_employment_status",
    "missing_height"
  ),
  name = "Missing Pattern",

  base_annotations = list(
    'Intersection size' =
      intersection_size(text = list(size = 3)) +
      labs(y = "Intersection size") +
      ggtitle("Patterns of Missing Data Across Patient Characteristics") +
      theme(
        plot.title = element_text(
          face = "bold",
          hjust = 0.5,
          margin = margin(b = 15)
        ),
        plot.margin = margin(t = 5, r = 5, b = 5, l = 0)
      )
  ),

  set_sizes =
    upset_set_size() +
    labs(caption = "Total missing per variable") +
    theme(
      plot.caption = element_text(
        hjust = 0.5,
        vjust = 7,
        size = 11
      ),
      axis.title.x = element_blank(),
      axis.title.y = element_blank()
```

```r
    )
) +
  labs(x = "Missingness Combination") +
  theme(
    axis.title.x = element_text(margin = margin(t = 15))
  )

# ----------------------------------------------------------
# 5. Missingness by clinical team
# ----------------------------------------------------------
# Calculate percentage missing per variable within each team.

missing_by_team <- data %>%
  group_by(clinical_team) %>%
  summarise(
    alcohol_missing = mean(is.na(alcohol_units)) * 100,
    height_missing  = mean(is.na(height)) * 100,
    weight_missing  = mean(is.na(weight)) * 100,
    employment_missing = mean(is.na(employment_status)) * 100
  )

print("===== MISSINGNESS BY CLINICAL TEAM (%) =====")
print(missing_by_team)

# Set clinical_team levels to numerical order (DN01-DN12)
# Ensures consistent ordering in plots and summaries.

data$clinical_team <- factor(
  data$clinical_team,
  levels = c("DN01","DN02","DN03","DN04","DN05","DN06",
             "DN07","DN08","DN09","DN010","DN011","DN012")
)


# Plot alcohol missingness proportion by clinical team
ggplot(data, aes(x = clinical_team, fill = is.na(alcohol_units))) +
  geom_bar(position = "fill") +
  labs(
    title = "Alcohol Intake Missingness by Clinical Team",
    x = "Clinical Team",
    y = "Proportion",
    fill = "Alcohol Missing"
  ) +
  theme(
    plot.title = element_text(
      face = "bold",
      hjust = 0.5,               # center title
```

```r
      margin = margin(b = 15)    # space below title
    ),
    axis.title.x = element_text(
      margin = margin(t = 15)    # space between x label and plot
    ),
    axis.title.y = element_text(
      margin = margin(r = 15)    # space between y label and plot
    )
  )

# ----------------------------------------------------------
# 6. Assessment of missingness mechanism
# ----------------------------------------------------------
# Evaluate associations between missingness indicators and
# screening behaviour or demographic variables.

# --------- Alcohol missing indicator ----------
data$alcohol_missing <- is.na(data$alcohol_units)

# Association with screening decision
print("===== CHI-SQUARE: Alcohol Missing vs Screening =====")
print(table(data$screening_decision, data$alcohol_missing))
print(chisq.test(data$screening_decision, data$alcohol_missing))

# Association with age
print("===== T-TEST: Age vs Alcohol Missing =====")
print(t.test(age ~ alcohol_missing, data = data))

# ------ Height missing indicator ------
data$height_missing <- is.na(data$height)

print("===== FISHER'S EXACT TEST: Height Missing vs Screening decision =====")
tab_height <- table(data$screening_decision, data$height_missing)
print(tab_height)
print(fisher.test(tab_height))

print("===== T-TEST: Age vs Height Missing =====")
print(t.test(age ~ height_missing, data = data))

# ---------- Employment missing indicator -------------
data$employment_missing <- is.na(data$employment_status)

print("===== CHI-SQUARE: Employment Missing vs Screening decision =====")
print(table(data$screening_decision, data$employment_missing))
print(chisq.test(data$screening_decision, data$employment_missing))

# ----------------------------------------------------------
```

```r
# 7. Visual comparison of age distribution by missingness
# ---------------------------------------------------------
# Compare age density distributions between individuals
# with and without missing alcohol intake data.

ggplot(data, aes(x = age, fill = alcohol_missing)) +
  geom_density(alpha = 0.4) +
  labs(
    title = "Age Distribution by Alcohol Missingness",
    x = "Age (years)",
    y = "Density",
    fill = "Alcohol Missing"
  ) +
  theme(
    plot.title = element_text(
      face = "bold",
      hjust = 0.5,              # center title
      margin = margin(b = 15)   # space below title
    ),
    axis.title.x = element_text(
      margin = margin(t = 15)   # space between x label and panel
    ),
    axis.title.y = element_text(
      margin = margin(r = 15)   # space between y label and panel
    )
  )


# ---------------------------------------------------------
# 8. Association Between Missingness and Clinical Team
# ---------------------------------------------------------
# Test whether missingness varies systematically across
# clinical teams using chi-square tests.

test_missing_vs_team <- function(var_name) {

  missing_indicator <- is.na(data[[var_name]])

  cat("\n====================================\n")
  cat("Missingness vs Clinical Team:", var_name, "\n")
  cat("====================================\n")

  tab <- table(data$clinical_team, missing_indicator)
  print(tab)

  # Chi-square test for 12x2 contingency table
  print(chisq.test(tab))
```

```r
}


# Run tests for selected variables
test_missing_vs_team("alcohol_units")
test_missing_vs_team("height")
test_missing_vs_team("employment_status")

# ----------------------------------------------------------
# 9. Summary Statistics by Clinical Team (Continuous Variables)
# ----------------------------------------------------------
# Calculate mean, SD, median, and IQR for numeric variables
# within each clinical team.

team_summary <- data %>%
  group_by(clinical_team) %>%
  summarise(
    n = n(),
    across(
      c(age, weight, height, alcohol_units, n_dependents),
      list(
        mean = ~mean(.x, na.rm = TRUE),
        sd = ~sd(.x, na.rm = TRUE),
        median = ~median(.x, na.rm = TRUE),
        IQR = ~IQR(.x, na.rm = TRUE)
      ),
      .names = "{.col}_{.fn}"
    )
  )

team_summary

# ----------------------------------------------------------
# 10. Distribution of Continuous Variables by Clinical Team
# ----------------------------------------------------------
# Visual inspection of within-team distributions using
# faceted histograms.

plot_histogram_by_team_facet <- function(df, var_name, bins = 20) {

  ggplot(df, aes(x = .data[[var_name]])) +
    geom_histogram(
      bins = bins,
      fill = "steelblue",
      colour = "black",
      na.rm = TRUE
    ) +
```

```r
    facet_wrap(~ clinical_team) +
    labs(
      title = paste("Distribution of", var_name, "by Clinical Team"),
      x = var_name,
      y = "Frequency"
    ) +
    theme_minimal() +
    theme(
      plot.title = element_text(face = "bold", hjust = 0.5)
    )
}

numeric_vars <- c("age", "weight", "height", "alcohol_units")

for (var in numeric_vars) {
  print(plot_histogram_by_team_facet(data, var))
}

# ------------------------------------------------------------
# 11. Descriptive Analysis of Categorical & Discrete Variables
# ------------------------------------------------------------

# Employment status distribution within each team
employment_by_team <- data %>%
  count(clinical_team, employment_status) %>%
  group_by(clinical_team) %>%
  mutate(
    percent = round(100 * n / sum(n), 1)
  ) %>%
  ungroup()

employment_by_team

ggplot(data, aes(x = employment_status)) +
  geom_bar(fill = "steelblue", colour = "black") +
  facet_wrap(~ clinical_team) +
  labs(
    title = "Employment Status Distribution by Clinical Team",
    x = "Employment Status",
    y = "Count"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    axis.text.x = element_text(angle = 45, hjust = 1)
  )
```

```r
# Screening decision distribution within each team
screening_by_team <- data %>%
  count(clinical_team, screening_decision) %>%
  group_by(clinical_team) %>%
  mutate(
    percent = round(100 * n / sum(n), 1)
  ) %>%
  ungroup()

screening_by_team

ggplot(data, aes(x = screening_decision)) +
  geom_bar(fill = "steelblue", colour = "black") +
  facet_wrap(~ clinical_team) +
  labs(
    title = "Screening Decision by Clinical Team",
    x = "Screening Decision",
    y = "Count"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5)
  )


# Number of dependents distribution within each team
dependents_by_team <- data %>%
  count(clinical_team, n_dependents) %>%
  group_by(clinical_team) %>%
  mutate(
    percent = round(100 * n / sum(n), 1)
  ) %>%
  ungroup()

dependents_by_team

# Bar chart: Distribution of number of dependents by clinical team
# Treated as categorical for clearer visual comparison
ggplot(data, aes(x = factor(n_dependents))) +
  geom_bar(fill = "steelblue", colour = "black") +
  facet_wrap(~ clinical_team) +
  labs(
    title = "Number of Dependents by Clinical Team",
    x = "Number of Dependents",
    y = "Count"
  ) +
  theme_minimal() +
```

```r
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5)
  )

# ----------------------------------------------------------
# 12. Between-Team Heterogeneity (Continuous Variables)
# ----------------------------------------------------------
# Assess differences across clinical teams using ANOVA
# and non-parametric testing where appropriate.

continuous_vars <- c("age", "weight", "height", "alcohol_units")

anova_results <- lapply(continuous_vars, function(var) {
  formula <- as.formula(paste(var, "~ clinical_team"))
  summary(aov(formula, data = data))
})

names(anova_results) <- continuous_vars
anova_results

# Kruskal-Wallis test for alcohol intake across clinical teams
# as it appears moderately right-skewed
kruskal_alcohol <- kruskal.test(alcohol_units ~ clinical_team, data = data)

kruskal_alcohol

# Height showed significant between-team heterogeneity (ANOVA p < 0.001).
# DN012 had a notably higher mean height compared to other teams (~1.75m vs ~1.60m),
# raising the possibility of a unit or scaling inconsistency.
# Raw DN012 values are inspected to verify measurement units
# and rule out harmonisation errors.

# Raw DN012 heights
summary(read.csv("raw_data/DN012.csv")$Height)

# Cleaned DN012 heights
data %>%
  filter(clinical_team == "DN012") %>%
  summarise(min = min(height, na.rm = TRUE),
            max = max(height, na.rm = TRUE),
            mean = mean(height, na.rm = TRUE))

# Raw DN012 heights were recorded in centimetres and correctly converted
# to metres during cleaning. No scaling inconsistencies detected.
# Observed between-team heterogeneity in height reflects genuine
# demographic variation rather than a data processing error.
```

```r
# Effect size (eta-squared) for height differences
model_height <- aov(height ~ clinical_team, data = data)
eta_height <- eta_squared(model_height)
eta_height

# ---------------------------------------------------------
# 13. Between-Team Heterogeneity (Categorical Variables)
# ---------------------------------------------------------
# Use chi-square tests to assess variation across teams.

# Exclude DN01 (employment not recorded) and drop unused factor levels
# to ensure valid chi-square testing.
data_noDN01 <- droplevels(
  data[data$clinical_team != "DN01", ]
)

# Contingency table
table_employment <- table(
  data_noDN01$employment_status,
  data_noDN01$clinical_team
)
table_employment

# Chi-square test
chisq.test(table_employment)

# Contingency table
table_screening <- table(data$screening_decision, data$clinical_team)
table_screening

# Chi-square test
chisq.test(table_screening)


# Contingency table
table_dependents <- table(data$n_dependents, data$clinical_team)
table_dependents

# Chi-square test
chisq.test(table_dependents)


# Although screening uptake showed statistical heterogeneity across teams,
# effect size is required to assess the practical magnitude of this association.
# Cramér's V provides a standardized measure of association for categorical data.
cramers_v(table_screening)
```

```r
# Calculate within-team screening proportions to examine differences
# in uptake rates across clinical teams.
prop.table(table_screening, margin = 2)

# ---------------------------------------------------------
# 14. Screening Uptake by Clinical Team
# ---------------------------------------------------------
# Estimate screening proportions with Wilson 95% confidence intervals.

screening_plot_data <- data %>%
  group_by(clinical_team) %>%
  summarise(
    total_n = n(),
    n_positive = sum(screening_decision == "Y"),
    prop_screened = n_positive / total_n,
    .groups = "drop"
  )

ci <- binom.confint(
  screening_plot_data$n_positive,
  screening_plot_data$total_n,
  methods = "wilson"
)

screening_plot_data$lower_ci <- ci$lower
screening_plot_data$upper_ci <- ci$upper

# Calculate overall mean screening acceptance across teams
overall_mean <- mean(screening_plot_data$prop_screened)

# Preserve ascending order and include sample size in x-axis label
screening_plot_data <- screening_plot_data %>%
  arrange(clinical_team) %>%
  mutate(
    team_label = paste0(clinical_team, "\n(n=", total_n, ")"),
    team_label = factor(team_label, levels = team_label)
  )

# Create bar plot of screening uptake by clinical team
ggplot(screening_plot_data, aes(x = team_label, y = prop_screened)) +

  # Bars representing screening acceptance per team
  geom_col(fill = "steelblue") +

  # Add 95% binomial confidence intervals
  geom_errorbar(
    aes(ymin = lower_ci, ymax = upper_ci),
```

```r
  width = 0.2,
  linewidth = 0.4
) +

# Add percentage labels above bars
geom_text(
  aes(label = paste0(round(prop_screened * 100, 1), "%")),
  vjust = -0.3,
  size = 3.5
) +

# Add dashed horizontal line for overall mean acceptance
geom_hline(
  yintercept = overall_mean,
  linetype = "dashed",
  colour = "darkred",
  linewidth = 0.8
) +

annotate(
  "text",
  x = -Inf,
  y = overall_mean,
  label = paste0("Mean: ",
                 round(overall_mean * 100, 1), "%"),
  hjust = 0,
  vjust = -0.5,
  colour = "darkred",
  size = 3.5
) +

scale_y_continuous(
  labels = scales::percent_format(accuracy = 1),
  limits = c(0, 1)
) +

labs(
  title = "Screening Acceptance Across Clinical Teams",
  x = "Clinical Team (Sample Size)",
  y = "Screening Acceptance (%)"
) +

theme_minimal() +
theme(
  plot.title = element_text(face = "bold", hjust = 0.5),
  axis.text.x = element_text(angle = 45, hjust = 1),
  axis.title.x = element_text(margin = margin(t = 12)),
```

```r
    axis.title.y = element_text(margin = margin(r = 12))
  )
```

0.8 Statistical Modelling (05_statistical_modelling.R)

```r
# ==========================================================
# Script: 05_statistical_modelling.R
# Purpose: Examine associations between patient
#          characteristics and breast cancer screening uptake
#          using univariate tests and multivariable logistic regression.
# Author: Nikolaos Samperis
# Date: 20/02/2026
# ==========================================================

# ----------------------------------------------------------
# 1. Load required libraries
# ----------------------------------------------------------
# tidyverse: data manipulation and visualisation
# broom: tidy model outputs for plotting
# car: variance inflation factor (VIF) for multicollinearity assessment

library(tidyverse)
library(broom)
library(car)

# ---------------------------------------------------------------------------
# 2. Unadjusted Associations Between Patient Characteristics and Screening
# ---------------------------------------------------------------------------

# Objective:
# Examine crude (unadjusted) associations between individual
# patient-level variables and screening uptake.
# Continuous variables are tested using t-tests (or Wilcoxon for skewed data),
# and categorical variables using chi-square tests.

# Null hypothesis (for each test):
# No association between the variable and screening uptake.

t.test(age ~ screening_decision, data = data)

# Non-parametric alternative due to right-skewed alcohol distribution
wilcox.test(alcohol_units ~ screening_decision, data = data)

chisq.test(table(data$employment_status, data$screening_decision))
chisq.test(table(data$n_dependents, data$screening_decision))
```

```r
t.test(weight ~ screening_decision, data = data)
t.test(height ~ screening_decision, data = data)

# ---------------------------------------------------------------------------
# 3. Multivariable Logistic Regression: Factors Associated with Screening
# ---------------------------------------------------------------------------

# Objective: assess independent predictors of screening uptake while adjusting
# for key patient characteristics and clinical team. A parsimonious model is
# specified a priori based on plausibility and interpretability, then a more
# comprehensive model is explored as a sensitivity analysis.

# Set reference level for screening outcome (optional but recommended)
# Here, "N" (not screened) is treated as the reference category
data$screening_decision <- relevel(data$screening_decision, ref = "N")

# ---------------------------------
# 3.1 Primary (parsimonious) model
# ---------------------------------
# Core covariates were selected based on their plausible relationship with
# screening behaviour and potential to confound team comparisons (age,
# employment status, number of dependents, alcohol intake),
# alongside clinical team.

model_primary <- glm(
  screening_decision ~
    age +
    alcohol_units +
    employment_status +
    n_dependents +
    clinical_team,
  data = data,
  family = binomial
)

summary(model_primary)

# Convert log-odds coefficients to Odds Ratios with 95% CI
exp(cbind(
  OR = coef(model_primary),
  confint(model_primary)
))

# ---- Multicollinearity Check ----
# VIF > 5 indicates moderate concern
# VIF > 10 indicates serious multicollinearity
```

```r
vif_primary <- vif(model_primary)
print(vif_primary)

# ------------------------------------------------------------------
# 3.2 Sensitivity analysis: extended model including weight & height
# ------------------------------------------------------------------
# Weight and height were associated with screening in univariate analyses.
# They are added here to assess whether these crude associations persist after
# adjustment, and whether inclusion materially changes the primary model estimates.

model_extended <- glm(
  screening_decision ~
    age +
    weight +
    height +
    alcohol_units +
    employment_status +
    n_dependents +
    clinical_team,
  data = data,
  family = binomial
)

summary(model_extended)

exp(cbind(
  OR = coef(model_extended),
  confint(model_extended)
))

# If the extended model shows inflated standard errors/unstable estimates
# (suggesting multicollinearity) and does not improve interpretability, the
# primary model is retained for final inference.

# ---- Multicollinearity Check (Extended Model) ----

vif_extended <- vif(model_extended)
print(vif_extended)

# -------------------------------------------------------------------------
# 3.3 Forest Plot - Adjusted Odds Ratios (Primary Model)
# -------------------------------------------------------------------------

# Tidy model output and exponentiate coefficients to OR scale
forest_data <- tidy(model_primary, conf.int = TRUE, exponentiate = TRUE) %>%
  filter(term != "(Intercept)") %>%
  mutate(
```

```r
    term_clean = case_when(
      term == "age" ~ "Age (per year increase)",
      term == "alcohol_units" ~ "Alcohol intake (per additional unit)",
      term == "n_dependents" ~ "Number of dependents",
      str_detect(term, "^employment_status") ~
        paste0(str_remove(term, "^employment_status"), " vs Employed"),
      str_detect(term, "^clinical_team") ~
        paste0(str_remove(term, "^clinical_team"),
               " vs DN01 (reference)"),
      TRUE ~ term
    ),
    group = if_else(str_detect(term, "^clinical_team"),
                    "Clinical Team",
                    "Patient Characteristics")
  )

# Order clinical teams numerically
teams_df <- forest_data %>%
  filter(group == "Clinical Team") %>%
  mutate(
    team_number = as.integer(str_extract(term_clean, "(?<=DN)\\d+"))
  ) %>%
  arrange(team_number) %>%
  mutate(term_clean = factor(term_clean, levels = rev(term_clean))) %>%
  select(-team_number)

# Order Patient Characteristics (keep logical order)
patients_df <- forest_data %>%
  filter(group == "Patient Characteristics") %>%
  mutate(term_clean = factor(term_clean, levels = rev(term_clean)))

# Combine
forest_data_final <- bind_rows(teams_df, patients_df)

# Forest plot of adjusted ORs (log scale)
ggplot(forest_data_final,
       aes(x = estimate, y = term_clean)) +
  geom_errorbarh(aes(xmin = conf.low, xmax = conf.high),
                 height = 0.2,
                 linewidth = 0.8) +
  geom_point(size = 3) +
  geom_vline(xintercept = 1,
             linetype = "dashed",
             colour = "grey40") +
  scale_x_log10() +
  facet_wrap(~ group, scales = "free_y") +
  labs(
```

```r
    x = "Adjusted Odds Ratio",
    y = "",
    title = "Factors Associated with Breast Cancer Screening Uptake",
    subtitle = "Values to the right of 1 indicate higher screening uptake"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5, margin = margin(b = 10)),
    plot.subtitle = element_text(hjust = 0.5, margin = margin(b = 15)),
    strip.text = element_text(face = "bold"),
    axis.text.y = element_text(size = 10),
    axis.title.x = element_text(margin = margin(t = 15))
  )
```